Learning Plaintext–Ciphertext Relations using Machine Learning

Neural Cryptanalysis of SPECK 32/64

Sweta Snigdha (2022527) Md Kaif (2022289)

March 2026

Abstract

We investigate the application of neural networks as differential distinguishers for the lightweight block cipher SPECK 32/64. Given ciphertext pairs generated from plaintexts with a fixed XOR difference, we train three architectures—a Multi-Layer Perceptron (MLP), a Convolutional Neural Network (CNN) with residual blocks, and a Siamese network—to distinguish reduced-round SPECK from a random permutation. Our experiments across 3 to 8 rounds show that all three models achieve near-perfect accuracy at 3 rounds ($\geq$99.4%), with the CNN achieving 99.99%. Distinguishing capability degrades gracefully with increasing rounds, reaching random-guess levels at 7 rounds. The CNN with residual blocks consistently outperforms the other architectures, achieving 87.7% accuracy at 5 rounds and 69.5% at 6 rounds.

1 Introduction

Classical cryptanalysis techniques such as differential and linear cryptanalysis exploit structured statistical relations between plaintexts and ciphertexts to distinguish a cipher from a random permutation. Differential cryptanalysis, introduced by Biham and Shamir [2], examines how input differences propagate through a cipher's rounds to produce output differences with non-uniform probability distributions.

Recent work by Gohr [1] demonstrated that neural networks can *automatically* learn such differential properties without being explicitly provided with cryptanalytic rules. A trained neural network can serve as a distinguisher: given a ciphertext pair, it predicts whether the pair was generated by a real cipher (with a specific input difference) or by a random permutation.

In this project, we apply this neural cryptanalysis framework to SPECK 32/64, a lightweight block cipher from the Simon/Speck family. We implement and compare three neural architectures—MLP, CNN, and Siamese networks—and study how distinguishing accuracy degrades as the number of cipher rounds increases.

2 Cryptographic Setup

2.1 SPECK 32/64

SPECK is a family of lightweight block ciphers designed by the NSA for resource-constrained environments. SPECK 32/64 operates on 32-bit blocks (two 16-bit words) with a 64-bit key (four 16-bit words) over 22 rounds.

Each round applies the following operations on words (x, y) with round key k_i:

$$x \leftarrow ((x \ggg 7) + y) \oplus k_i \tag{1}$$

$$y \leftarrow (y \lll 2) \oplus x \tag{2}$$

where $\ggg$ and $\lll$ denote right and left circular rotation on 16-bit words, and $\oplus$ denotes XOR.

The key schedule expands the 64-bit key into 22 round keys using the same round function structure, making the cipher efficient to implement and analyze.

2.2 Differential Distinguishing

Let F be an unknown function that is either:

(a) A reduced-round SPECK cipher E_k, or

(b) A uniformly random permutation π.

Given plaintext pairs $(P, P \oplus \Delta P)$ and their corresponding outputs $(C, C') = (F(P), F(P \oplus \Delta P))$, the task is to determine whether F is a real cipher or random. A real cipher produces output differences $\Delta C = C \oplus C'$ with a non-uniform distribution (due to the differential characteristics of the cipher), while a random permutation produces uniformly distributed ΔC.

We use the input difference $\Delta P = (0\text{x}0040, 0\text{x}0000)$, following Gohr [1]. This difference flips a single bit in the left word and leaves the right word unchanged, producing exploitable differential trails in reduced-round SPECK.

3 Dataset Generation

3.1 Labeling Scheme

Each sample is a ciphertext pair (C, C') with a binary label:

$$y = \begin{cases} 1 & \text{cipher-generated} \\ 0 & \text{random-generated} \end{cases} \tag{3}$$

3.2 Generation Process

For **positive samples** $(y = 1)$:

1. Sample a random plaintext $P \in \{0,1\}^{32}$
2. Compute $P' = P \oplus \Delta P$
3. Sample a random key $K \in \{0,1\}^{64}$
4. Encrypt: $(C, C') = (E_K(P),\ E_K(P'))$ using r rounds

For **negative samples** $(y = 0)$:

1. Sample two independent random values $C, C' \in \{0,1\}^{32}$

The dataset is balanced with an equal number of positive and negative samples. We use 500,000 training samples and 100,000 test samples for each round configuration, with fresh random keys per sample.

3.3 Input Representation

We use the **raw ciphertext pair** representation: each ciphertext is decomposed into individual bits (MSB first), and the two 32-bit ciphertexts are concatenated to form a 64-dimensional binary feature vector:

$$\mathbf{x} = (C_0[31], C_0[30], \ldots, C_0[0],\ C_1[31], C_1[30], \ldots, C_1[0]) \in \{0,1\}^{64} \tag{4}$$

This representation preserves the full information of both ciphertexts and allows the models to learn any function of the pair, including the XOR difference ΔC as well as individual bit patterns.

4 Machine Learning Models

We implement three architectures of increasing structural bias toward the distinguishing task.

4.1 Multi-Layer Perceptron (MLP)

The MLP is a baseline fully-connected architecture with four hidden layers:

Layer	Output Dimension
Linear + BatchNorm + ReLU + Dropout(0.1)	256
Linear + BatchNorm + ReLU + Dropout(0.1)	256
Linear + BatchNorm + ReLU + Dropout(0.1)	128
Linear + BatchNorm + ReLU + Dropout(0.1)	64
Linear + Sigmoid	1

Total parameters: ∼115K. The MLP treats input bits as an unstructured vector with no assumptions about the relationship between C and C'.

4.2 Convolutional Neural Network (CNN)

Our CNN reshapes the 64-bit input into a 2-channel signal—one channel for each ciphertext's 32 bits—enabling the convolutional layers to detect local bit-pattern correlations *within and across* ciphertexts:

Layer	Output Shape
Reshape input to $(2, 32)$	2×32
Conv1d($2 \to 64$, k=3) + BatchNorm + ReLU	64×32
Residual Block (2× Conv1d, skip connection)	64×32
Residual Block (2× Conv1d, skip connection)	64×32
Flatten	2048
Linear + BatchNorm + ReLU + Dropout(0.2)	256
Linear + BatchNorm + ReLU	64
Linear + Sigmoid	1

Each residual block contains two convolutional layers with a skip connection:

$$\mathbf{h}' = \text{ReLU}(\mathbf{h} + \text{Conv}(\text{ReLU}(\text{BN}(\text{Conv}(\mathbf{h})))))) \tag{5}$$

The 2-channel design is crucial: by separating C and C' into distinct channels, the initial convolution learns cross-ciphertext correlations from the first layer.

4.3 Siamese Network

The Siamese architecture processes each ciphertext through a shared-weight branch before combining:

Component	Architecture
Shared Branch (applied to C and C' separately)	Linear(32→128) + BN + ReLU Linear(128→64) + BN + ReLU
Combiner (input: $[\mathbf{e}_0;\ \mathbf{e}_1;\ \lvert\mathbf{e}_0 - \mathbf{e}_1\rvert]$)	Linear(192→128) + BN + ReLU + Drop(0.2) Linear(128→64) + BN + ReLU Linear(64→1) + Sigmoid

Total parameters: ∼50K. The shared branch encodes a "ciphertext embedding," and the combiner receives the concatenation of both embeddings plus their absolute difference $|\mathbf{e}_0 - \mathbf{e}_1|$, which provides an explicit learned-difference signal.

5 Training Setup

All models are trained with the following configuration:

- **Loss function:** Binary Cross-Entropy (BCE)
- **Optimizer:** Adam with learning rate 10^{-3} and weight decay 10^{-5}
- **Scheduler:** Cosine annealing over 40 epochs
- **Batch size:** 5,000
- **Early stopping:** Patience of 7 epochs on test accuracy
- **Hardware:** NVIDIA H100 PCIe (MIG 3g.40gb partition)

Each (model, round) combination is trained independently with freshly generated data. Seeds are fixed for reproducibility.

6 Experimental Results

6.1 Accuracy vs. Rounds

Table 1 and Figure 1 show the test accuracy of each model across 3–8 rounds of SPECK 32/64.

Table 1: Test accuracy (%) of each distinguisher across round counts.

Model	r=3	r=4	r=5	r=6	r=7	r=8
MLP	99.37	92.45	82.30	60.36	50.23	50.32
CNN	**99.99**	**97.37**	**87.72**	**69.55**	50.24	50.12
Siamese	99.40	92.82	83.03	68.59	50.40	50.27

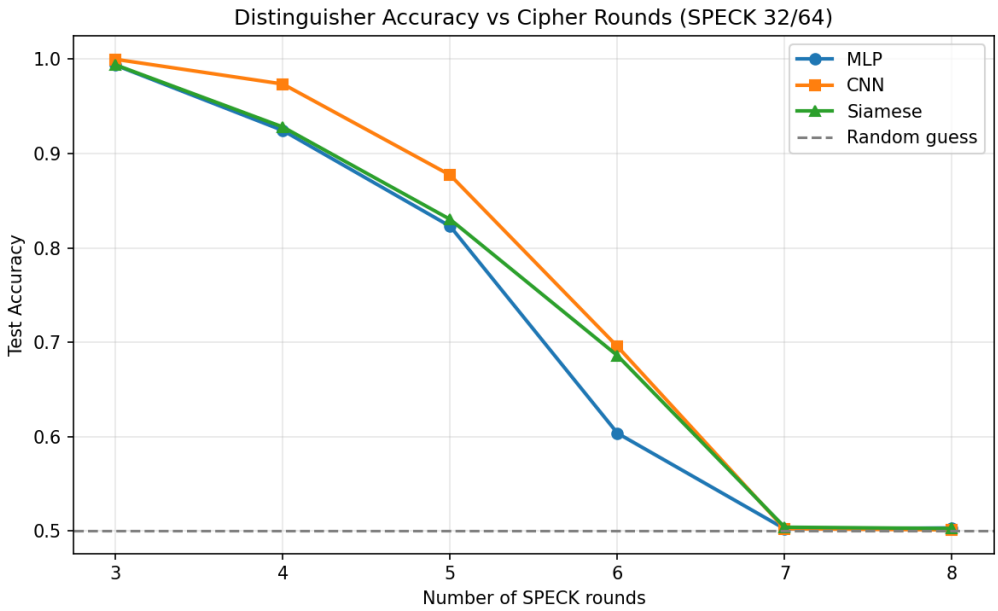


Figure 1: Test accuracy vs. number of SPECK rounds for all three models. The dashed line indicates random-guess performance (50%).

6.2 Key Observations

1. **CNN dominates across all distinguishable rounds.** The residual CNN achieves near-perfect accuracy (99.99%) at 3 rounds and maintains meaningful distinguishing power up to 6 rounds (69.55%). The 2-channel input design and residual connections enable it to capture both local bit-pattern correlations and cross-ciphertext dependencies.

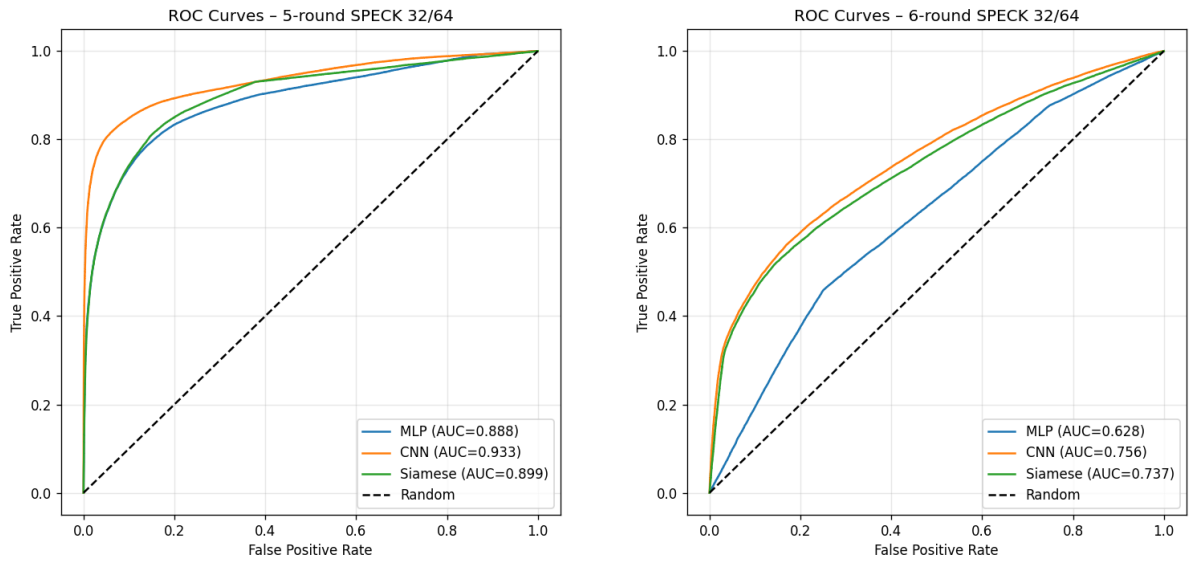
2. **Siamese outperforms MLP at higher rounds.** While MLP and Siamese perform similarly at 3–4 rounds, the Siamese network's structural bias—shared-weight branches with an explicit difference signal—provides an advantage at 6 rounds (68.59% vs. 60.36%).

3. **All models converge to 50% at 7 rounds.** Beyond 6 rounds, no model achieves accuracy significantly above random guessing. This indicates that the differential bias from $\Delta P = (0\text{x}0040, 0\text{x}0000)$ is fully diffused after 7 rounds of SPECK.

4. **Maximum distinguishable rounds: 6.** The CNN maintains $\sim$70% accuracy at 6 rounds, establishing this as the boundary of neural distinguishing for SPECK 32/64 with our chosen input difference. SPECK 32/64 uses 22 rounds in its full specification, providing a security margin of $22 - 6 = 16$ rounds against this attack.

6.3 ROC Curves

Figures 2a and 2b show ROC curves for the 5-round and 6-round cases, where the separation between models is most apparent.



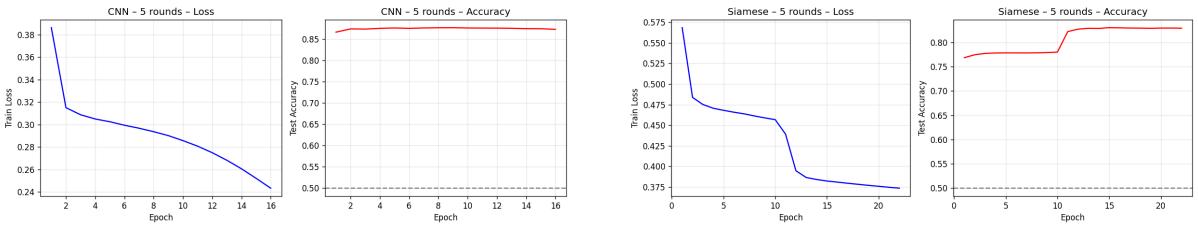
(a) 5-round SPECK (b) 6-round SPECK

Figure 2: ROC curves comparing all three models at 5 and 6 rounds.

6.4 Training Dynamics

Figure 3 shows selected training curves. Notable patterns include:

- The CNN converges rapidly at low rounds (12 epochs at 3 rounds) but struggles at 7+ rounds where the loss decreases without accuracy improvement, indicating overfitting to noise.
- The Siamese network exhibits a characteristic "phase transition" during training—accuracy plateaus before a sudden jump (e.g., epoch 11 at 5 rounds), suggesting that the shared branch learns a useful embedding before the combiner discovers how to exploit it.
- The MLP shows the most stable but slowest convergence, consistent with its lack of structural bias.



(a) CNN at 5 rounds (b) Siamese at 5 rounds

Figure 3: Training curves showing loss and test accuracy over epochs.

7 Discussion

7.1 Why CNN Outperforms

The CNN's superiority stems from two design choices:

1. **2-channel input:** Presenting C and C' as separate channels allows the first convolutional layer to directly compute cross-ciphertext features, effectively learning ΔC and higher-order correlations simultaneously.

2. **Residual connections:** Skip connections prevent gradient degradation and allow the network to learn identity mappings where no useful transformation exists, making the architecture robust across varying difficulty levels.

An earlier CNN architecture using single-channel input with global average pooling achieved only $\sim$51% accuracy even at 5 rounds, confirming that the input structure and architecture choices are critical.

7.2 Relationship to Classical Cryptanalysis

The decay in accuracy from 3 to 7 rounds mirrors the decay in differential probability with increasing rounds. In classical differential cryptanalysis, the probability that a differential trail holds decreases exponentially with the number of rounds. Our neural models implicitly learn these differential characteristics: at low rounds, the non-uniform distribution of ΔC is statistically detectable; at higher rounds, ΔC approaches a uniform distribution and becomes indistinguishable from random.

The transition at 7 rounds aligns with the known differential properties of SPECK 32/64, where the best classical differential distinguishers also lose effectiveness in this range for single-trail attacks.

7.3 Limitations

- We use a single fixed input difference ΔP. Searching over multiple input differences or combining results across differences could extend the distinguishable round count.

- Our models are purely black-box distinguishers. Interpretability techniques (e.g., gradient analysis) could reveal what differential features the networks learn.

- We do not attempt key recovery, which would require conditioning the distinguisher on candidate subkey values.

8 Conclusion

We demonstrated that neural networks can effectively learn plaintext–ciphertext relations in SPECK 32/64 and serve as differential distinguishers for up to 6 rounds. Among the three architectures evaluated, the residual CNN with 2-channel input achieved the best results across all distinguishable round counts, reaching 99.99% accuracy at 3 rounds and 69.55% at 6 rounds. All models fail at 7 rounds, establishing the boundary of neural distinguishing for this cipher and input difference.

These results confirm that ML-based cryptanalysis can automatically discover exploitable differential properties without explicit cryptanalytic rules, while the full 22-round SPECK 32/64 retains a substantial security margin against such attacks.

References

[1] A. Gohr, "Improving Attacks on Round-Reduced Speck32/64 Using Deep Learning," in *Advances in Cryptology – CRYPTO 2019*, Lecture Notes in Computer Science, vol. 11693, Springer, 2019, pp. 150–179.

[2] E. Biham and A. Shamir, "Differential Cryptanalysis of DES-like Cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1991.

[3] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK Families of Lightweight Block Ciphers," IACR Cryptology ePrint Archive, Report 2013/404, 2013.